SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

QUESTIONS: 5 Duration: 1 Hour Total Marks: 50

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.* ***(BL3)***

Assessment Tool Weightage: 50%

Annexure A

Analytical Problem Solving

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA17 — ARTIFICIAL INTELLIGENCE

Assessment Tool 1 — Analytical Problem Solving: Answers

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively

---

Q1. The Water Jug Problem

Two jugs — 4-gallon and 3-gallon, no markings — plus a pump. Goal: get exactly 2 gallons into the 4-gallon jug.

Problem Formulation

State: (x, y) — x = water currently in the 4-gallon jug (0–4), y = water in the 3-gallon jug (0–3).

Initial state: (0, 0) — both jugs empty.

Goal test: x = 2 (any y) — the 4-gallon jug contains exactly 2 gallons.

Operators (actions):

- Fill the 4-gallon jug from the pump: (x, y) → (4, y)
- Fill the 3-gallon jug from the pump: (x, y) → (x, 3)
- Empty the 4-gallon jug onto the ground: (x, y) → (0, y)
- Empty the 3-gallon jug onto the ground: (x, y) → (x, 0)
- Pour from the 3-gallon jug into the 4-gallon jug until either the 4-gallon jug is full or the 3-gallon jug is empty
- Pour from the 4-gallon jug into the 3-gallon jug until either the 3-gallon jug is full or the 4-gallon jug is empty

Solution Path (found by search over the state space)

Step	Action	4-gal jug	3-gal jug
0	Start	0	0
1	Fill the 3-gallon jug	0	3
2	Pour 3-gallon jug into 4-gallon jug	3	0
3	Fill the 3-gallon jug again	3	3
4	Pour 3-gallon jug into 4-gallon jug (until 4-gallon jug is full)	4	2
5	Empty the 4-gallon jug	0	2
6	Pour 3-gallon jug (2 gal) into 4-gallon jug	2	0

After step 6, the 4-gallon jug contains exactly 2 gallons — the goal state (2, 0) is reached in 6 actions.

Q2. Mars Rover — Agent Analysis

(a) Percepts

- Camera/visual images of terrain, rocks, and soil
- Spectrometer readings (mineral/chemical composition of samples)
- Position/orientation data (odometry, star-tracker, GPS-like localisation)
- Obstacle-sensor data (LIDAR / stereo-vision / ultrasonic range readings)
- Environmental sensors — temperature, radiation level, dust/atmospheric conditions
- Rover's own status — battery/power level, wheel/motor health, arm position
- Communication signals/commands relayed from mission control on Earth

(b) Characterisation of the Operating Environment

- Partially observable — sensors have limited range and can be obscured by dust storms or terrain shadows; the rover never has full knowledge of the planet.
- Single-agent (from the rover's perspective) — though it may coordinate with orbiters, it is the sole actor operating on the surface at a time.
- Stochastic/non-deterministic — wheel slippage, sensor noise, and unpredictable terrain mean the same action does not always lead to the same result.
- Sequential — each action (drive, drill, sample) affects future options via changed position, resources, and terrain state.
- Dynamic — the environment (lighting, dust storms, temperature) changes over time, including while the rover is deliberating.
- Continuous — position, sensor readings, and time are all continuous-valued.
- Unknown — the exact terrain map and sample locations are not known in advance and must be discovered through exploration.

(c) Actions the Agent Can Take

- Navigate/move (drive forward/backward, turn, climb over small obstacles)
- Avoid or re-route around hazards detected by obstacle sensors
- Extend/operate the robotic arm to reach a target
- Drill or scoop and collect a soil/rock sample
- Operate onboard instruments (camera, spectrometer) to analyse a sample or scene
- Store a sample onboard for later analysis or return
- Orient solar panels / manage power to recharge
- Transmit collected data/images back to Earth via orbiter relay

(d) Performance Evaluation

- Quantity and scientific value of samples successfully collected and analysed
- Accuracy/reliability of the scientific data gathered
- Distance safely traversed without damage, getting stuck, or tipping over
- Power/energy efficiency (useful work done per unit of battery consumed)
- Amount and quality of data successfully transmitted back to Earth
- Degree of autonomy achieved (less need for step-by-step human commands, important given communication delay to Mars)
- Overall mission longevity/lifespan without critical failure

(e) Most Suitable Agent Architecture

A **hybrid model-based, utility-based agent (with a learning component)** is the most suitable architecture, for these reasons:

- A simple reflex agent is not enough because the environment is only partially observable — the rover must maintain an internal model of terrain already mapped, its own position, and resource levels to make sensible decisions (model-based).
- A purely goal-based agent (reach a sample and drill it) is insufficient because the rover routinely faces competing objectives — e.g., drive further to a scientifically richer sample vs. conserve battery vs. transmit data now — which requires weighing trade-offs, i.e. a utility function over outcomes (utility-based).
- A learning element (as used in real missions, e.g. AEGIS autonomous target-selection on the Curiosity/Perseverance rovers) lets the rover improve terrain/hazard classification and target selection over time from experience, rather than relying only on rules programmed before launch.

Q3. The 8-Queens Problem — Search Space Formulation

Place 8 queens on a chessboard so that no two queens attack each other (share a row, column, or diagonal).

Incremental (Efficient) Problem Formulation

State: any arrangement of 0 to 8 queens on the board, with at most one queen per column, placed column-by-column from left to right.

Initial state: an empty board (no queens placed).

Actions/Successor function: add a queen to the left-most empty column, in any row of that column such that it is not attacked by any queen already placed (i.e., no shared row or diagonal with existing queens).

Transition model: returns the board with the new queen added at the chosen square.

Goal test: 8 queens are placed on the board and no two of them attack each other.

Path cost: not relevant here — every step costs the same (this is a constraint-satisfaction style problem where only the final configuration matters, not the path taken to reach it).

Why the Given Configuration Fails

In the figure, the queen placed in column 1 and the queen placed in column 8 sit on the same diagonal (their row and column indices differ by the same amount, e.g. row − column is constant along that diagonal). Since the goal test requires that no two queens share a row, column, or diagonal, this configuration violates the diagonal-attack constraint and is therefore rejected even though all 8 columns are filled.

Suitable Search Strategies

- Backtracking search (depth-first search with constraint-checking at each step) — places one queen per column and immediately backtracks whenever a placement conflicts with existing queens, avoiding the full $8^8 = 16{,}777{,}216$ raw search space by pruning early.
- Local search / Min-Conflicts algorithm — starts with all 8 queens placed (one per column, possibly conflicting) and repeatedly moves the queen with the most conflicts to the row with the fewest conflicts; this is dramatically faster in practice and scales to millions of queens, though it is not guaranteed to find a solution as systematically as backtracking.

Q4. OLA Cab Booking — Problem-Solving Agent Type and Pseudocode

A customer wants to travel between two locations and can choose among several cab types (mini, micro, sedan, shared, prime, etc.) based on comfort/cost preference.

Type of Problem-Solving Agent

This scenario is best modelled as a **Utility-Based Simple Problem-Solving Agent**. It has two layers of decision-making:

- Goal-based search — finding a route from the pickup location to the destination is a classic route-finding problem (states = locations/junctions, actions = road segments, goal test = reached destination), solvable with search algorithms such as A* or Uniform Cost Search over the road network.
- Utility-based selection — choosing which cab type to book is not a single fixed goal but a trade-off between competing factors (fare, waiting time, ride comfort, number of co-passengers for shared rides), which requires a utility function to rank the available options rather than simply satisfying a goal.

Pseudocode

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, the agent's current understanding of the world
              goal, initially null
              problem, a problem formulation

  state <- UPDATE-STATE(state, percept)      // current location, traffic, etc.
  if seq is empty then
      goal <- FORMULATE-GOAL(state)           // e.g., reach destination D
      problem <- FORMULATE-PROBLEM(state, goal)  // road network as graph
      seq <- SEARCH(problem)                  // e.g., A* / UCS finds route
      if seq = failure then return a null action
  action <- FIRST(seq)
```

```
  seq <- REST(seq)
  return action

function SELECT_CAB_OPTION(available_cabs, customer_preferences) returns best_cab
  for each cab in available_cabs:
      utility(cab) <- w1 * (1 / estimated_fare(cab))
                      + w2 * (1 / estimated_wait_time(cab))
                      + w3 * comfort_rating(cab)
  return the cab in available_cabs with the highest utility(cab)
```

The first routine finds the actual path to the destination; the second routine then ranks the mini/micro/sedan/shared/prime options by a weighted utility of fare, wait time, and comfort, and books the option with the highest utility for that customer.

Q5. Least-Cost Delivery Route — Uniform Cost Search (UCS)

Note: *The assignment text refers to "the delivery network represented as a weighted graph" below it, but the actual graph/diagram was not included in the uploaded file (only the question text came through). The method and full UCS procedure are explained below, worked through on a clearly-labelled illustrative example graph so you can see exactly how to apply it — swap in the real warehouse graph from your assignment sheet once you have it, and the same steps apply directly.*

Uniform Cost Search — Method

UCS is an uninformed search strategy that always expands the frontier node with the lowest cumulative path cost g(n) from the start node (not the fewest edges), making it optimal for weighted graphs such as a delivery network where edge costs represent fuel + time.

- Maintain a frontier as a priority queue ordered by cumulative path cost g(n).
- At each step, remove and expand the lowest-cost node from the frontier.
- For each neighbour, compute the new cumulative cost; if this is cheaper than any previously known cost to that neighbour, update it ("decrease-key") and record the new path.
- Stop when the goal node G is removed from the frontier for expansion — at that point the cheapest path has been found (UCS is optimal provided all edge costs are non-negative).

Illustrative Example Graph

Step-by-Step UCS Trace

Step	Node Expanded	Path So Far	Cumulative Cost	Frontier After Expansion
1	S	S	0	A(2), B(5)
2	A	S–A	2	B(3) via S-A-B [replaces B(5)], C(8) via S-A-C
3	B	S–A–B	3	C(5) via S-A-B-C [replaces C(8)], D(7) via S-A-B-D
4	C	S–A–B–C	5	D(7), G(8) via S-A-B-C-G
5	D	S–A–B–D	7	G(8) [G(9) via D is worse, not updated]
6	G (goal)	S–A–B–C–G	8	— search stops, goal expanded

Least-cost path found: **S → A → B → C → G** with total cost **2 + 1 + 2 + 3 = 8**. Note how UCS discovers a cheaper route to B (cost 3 via A) after already placing it in the frontier at cost 5 directly from S, and similarly finds a cheaper route to C (cost 5 via B) than its first-seen cost of 8 via A — this re-evaluation of already-frontiered nodes is exactly what guarantees UCS always returns the optimal path.